ECE6255 - Team 9 Report

Jack Newcomb, Anish Nair, Hao Ma

April 16, 2026

1 Introduction

The goal of this project is to develop a software system for modifying the duration of selected segments of a speech signal. The framework is designed to support user-specified edits in either of two forms: a scaling factor applied to a selected segment, or a desired target duration for that segment. In addition, the system supports multiple edits within a single audio file, and is intended to perform duration modification while preserving perceptually important speech characteristics as well as possible.

To support this goal, a modular processing pipeline was designed that loads an input audio signal, parses one or more requested edits, applies duration modification to the selected segments, and writes the resulting output signal. This structure separates the signal-processing algorithm from the surrounding application logic, making the system easier to test, validate, and extend.

For the core duration-modification stage, we selected the Pitch-Synchronous Overlap and Add (PSOLA) method. PSOLA is a well-known speech processing technique for time-scale modification because it allows changes in segment duration while better preserving pitch-related structure than naive resampling approaches. In this project, PSOLA serves as the primary algorithmic method for modifying speech duration within the broader editing framework.

2 System Overview

This project was implemented in Python using a modular design to separate responsibilities across the major stages of the processing pipeline. At a high level, the system takes an input speech waveform, accepts one or more user-specified duration edits, applies duration modification to the selected segments, and reconstructs a final edited output waveform. This organization was chosen to keep the signal-processing algorithm isolated from file handling, edit parsing, and reconstruction logic, making the framework easier to test and extend.

The system is composed of the following primary modules:

- **Audio I/O:** Responsible for loading input WAV files and writing processed output audio files.
- **Edit Specification Interface:** Handles the representation and parsing of user-requested edits, including start time, end time, and either a scaling factor or target duration.
- **Segment Extraction:** Converts time-based edit definitions into sample indices and extracts the corresponding waveform segments to be modified.
- **Duration-Scaling Interface:** Provides the interface between the general editing framework and the core duration-modification algorithm. In the final system, this module is responsible for applying PSOLA-based time-scale modification to the selected speech segment.
- **Stitching/Reconstruction:** Reinserts the modified segment into the original waveform while preserving the unedited regions of the signal.
- **Output Generation:** Produces the final edited waveform and any associated artifacts needed for inspection or analysis.

In addition to the core pipeline, supporting scripts were developed for post-processing and evaluation. These scripts assist with generating analysis outputs such as waveform and spectrogram comparisons, and help support the final assessment of system performance.

This modular structure made it possible to validate much of the end-to-end workflow independently of the final algorithm implementation. As a result, components such as file handling, edit validation, batch processing, and output generation could be developed and tested before the PSOLA stage was fully integrated.

3 Software Architecture and Design

This section provides a more detailed discussion of the software engineering decisions behind the implementation of each module. In addition to the underlying speech-processing objective, the project required a software structure that could support user-configurable edits, robust validation, modular testing, and straightforward integration of the core duration-modification algorithm. For this reason, the system was designed as a pipeline of relatively isolated modules with clearly defined responsibilities and interfaces.

At the application level, a main orchestration layer coordinates the flow between modules. It loads the input waveform, parses and validates requested edits, extracts the relevant segment, invokes the duration-scaling interface, reconstructs the waveform, and finally writes the edited output. This orchestration layer keeps module interactions explicit while avoiding unnecessary entanglement between user-facing control flow and lower-level processing functions.

3.1 Audio I/O

Audio input and output functionality was implemented in `audio_io.py`. This module is responsible for reading input `.wav` files into an internal array representation and writing the final processed waveform back to disk. Separating file handling from the remainder of the processing pipeline simplified both debugging and testing, since waveform manipulation could be developed independently of the details of file parsing.

The `load_wav` function loads a `.wav` file and performs extensive validation to guard against invalid inputs and common edge cases. These checks include file existence, invalid or unsupported file contents, empty audio data, and invalid sample rate conditions. By concentrating such validation in a single module, the remainder of the pipeline can operate under clearer assumptions about the structure and validity of the loaded signal.

The `save_wav` function handles writing the processed waveform to a `.wav` file. This provides a clean endpoint for the editing pipeline and ensures that the output stage is separated from the signal-processing logic itself.

3.2 Edit Specification Interface

The edit specification interface is responsible for representing and validating user-requested duration modifications before they are applied to the waveform. Conceptually, this layer serves as the boundary between the user-facing workflow and the lower-level signal-processing pipeline. Each requested edit specifies a time interval within the original signal together with either a duration scaling factor or a target output duration.

A key design goal of this interface was flexibility. The system was designed to support both single-edit and batch-edit workflows, allowing a user either to modify one segment directly or to provide multiple requested edits through a structured input format such as a CSV file. This made the framework more practical to use while preserving a consistent internal representation of edits once parsing was complete.

Validation was also an important responsibility of this module. Because downstream processing assumes well-formed edit requests, the edit interface performs checks such as interval consistency, nonnegative timing, and detection of overlapping edit regions in batch mode. This prevents invalid requests from propagating further into the processing pipeline and improves the overall robustness of the system.

3.3 Segment Extraction

Segment extraction was implemented in `segment.py`. This module is responsible for converting user-specified time intervals into the corresponding sample-index ranges and extracting the selected waveform segment from

the full audio signal. The extracted segment can then be passed directly to the duration-modification module for processing.

This separation was useful because it isolates the time-to-sample conversion logic from both the user interface layer and the signal-processing algorithm. In doing so, it reduces coupling and makes the duration-modification module easier to test independently, since that module can operate on a clean one-dimensional sample array rather than needing to manage edit parsing or file-level concerns.

The segment extraction stage also includes several validation checks, including checks for negative times, invalid interval definitions, and empty extracted sample ranges. The output of this module is a one-dimensional array of samples together with the implicit location information needed for later reinsertion into the original waveform.

3.4 Duration-Scaling Interface

The duration-scaling interface is the point at which the general editing framework connects to the core speech-processing algorithm. Its purpose is to accept an extracted waveform segment together with the requested edit parameters, apply duration modification, and return a modified segment suitable for reintegration into the original signal.

This abstraction was intentionally designed so that the surrounding application logic would remain independent of the specific implementation details of the underlying algorithm. In the final system, this module is responsible for applying the PSOLA-based duration-modification method. By isolating the algorithm behind a clear interface, the remainder of the system could be developed and tested before the final PSOLA implementation was completed.

3.5 Stitching/Reconstruction

Stitching and reconstruction are performed after the duration-modification stage returns a modified waveform segment. This functionality was implemented in `stitch.py`. The purpose of this module is to replace the original user-specified segment with the modified segment while preserving the portions of the waveform that were not edited.

This design keeps reconstruction logic separate from both extraction and duration modification. As a result, each stage of the pipeline performs a narrow and well-defined task: extraction isolates the region of interest, the algorithm modifies that region, and reconstruction reinserts the result into the full signal. This modularity improves readability and makes it easier to verify correctness at each stage.

3.6 Output Generation

Final output generation consists primarily of producing the edited waveform and associated analysis artifacts. The edited audio output is generated through the same audio I/O layer used for file writing, while additional post-processing scripts were developed for evaluation and inspection of results.

In particular, supporting scripts were used to generate artifacts such as waveform comparisons, spectrogram comparisons, and other plots relevant to qualitative and quantitative assessment. Although these evaluation tools are not part of the core editing pipeline itself, they form an important part of the overall project because they provide a structured way to assess the effectiveness of the implemented duration-modification method.

4 Configuration and Usage

The project was designed to support straightforward command-line use for both single-edit and batch-edit duration modification tasks. The implementation assumes a mono `.wav` input file and accepts user-specified edit parameters that determine which portion of the waveform should be modified and how the duration change should be applied.

4.1 Environment and Dependencies

The recommended execution environment for the project is Python 3.10 or later. The primary dependencies used by the framework are `numpy`, `scipy`, `matplotlib`, and `pytest`. These libraries support waveform manipulation, file I/O, visualization, and testing.

Dependencies may be installed with:

```
pip install numpy scipy matplotlib pytest
```

4.2 Input Assumptions

The system operates on mono `.wav` speech files. Each requested edit is specified by a start time t_1 and end time t_2, together with either a duration scaling factor or a target output duration. Internally, these time-domain edit requests are converted into sample-index ranges before segment extraction and processing.

The framework supports two modes of operation:

- **Single-edit mode**, in which one speech segment is modified at a time.
- **Batch-edit mode**, in which multiple edits are provided through a CSV specification file.

4.3 Single-Edit Usage

Single-edit mode is used when modifying one selected region of the input waveform. In this mode, the user specifies the input file, output file, time interval, and either a scale factor or a target duration.

A scale-factor-based edit may be invoked as follows:

```
python main.py examples/tsignal.wav --output examples/output.wav \
    --t1 0.20 --t2 0.40 --scale 1.5
```

This command stretches the region from 0.20 s to 0.40 s by a factor of 1.5.

Alternatively, a target duration may be specified directly:

```
python main.py examples/tsignal.wav --output examples/output.wav \
    --t1 0.20 --t2 0.40 --target-duration 0.50
```

In this case, the selected segment is modified so that its output duration is 0.50 s.

4.4 Batch-Edit Usage

Batch-edit mode allows multiple edits to be applied to the same input file using a CSV specification. This is useful when several independent regions of the waveform must be modified in a single run.

Batch mode may be invoked as follows:

```
python main.py examples/tsignal.wav --output examples/output.wav \
    --csv examples/sample_input.csv
```

The CSV file must include the columns `t1` and `t2`, along with at least one of `scale` or `target_duration`. Each row must specify exactly one of these two duration-modification options.

An example CSV file is:

```
t1,t2,scale,target_duration
0.20,0.40,1.5,
0.60,0.80,,0.30
```

In this example, the first segment is stretched by a factor of 1.5, while the second segment is modified so that its new duration is 0.30 s.

4.5 Batch Timing Convention and Validation

A key design choice in batch mode is that edit times are interpreted relative to the original input-file timeline rather than the progressively modified output timeline. This means that if an earlier edit changes the duration of one region, later edit times are still understood as referring to their intended locations in the original signal. Internally, the system remaps these later edit boundaries onto the current modified waveform as needed.

To preserve correctness and avoid ambiguity, overlapping edits in the original timeline are not supported. For example, the following CSV specification is invalid:

```
t1,t2,scale
0.20,0.50,1.2
0.40,0.70,0.8
```

because the second edit overlaps the first in the original signal.

4.6 Evaluation and Testing Workflow

In addition to the main processing pipeline, the repository includes a separate evaluation script for comparing original and modified audio outputs. Basic evaluation may be run with:

```
python scripts/evaluate.py --original examples/tsignal.wav \
    --modified examples/output.wav --report-dir reports/basic_eval
```

This generates waveform comparisons, spectrogram visualizations, and a basic metrics report. Region-focused evaluation is also supported by specifying the edited interval and a margin around it.

The project also includes an automated test suite, which may be run with:

```
pytest
```

These tests cover key framework functionality, including segment extraction, stitching behavior, CSV parsing, batch processing, target-duration handling, remapping of batch edit timelines, and rejection of overlapping edits.

5 Duration Modification Algorithm

The core duration-modification method used in this project is a time-domain Pitch-Synchronous Overlap and Add (TD-PSOLA) approach. PSOLA is commonly used for speech time-scale modification because it changes segment duration by reusing short waveform regions aligned to the local pitch structure, which typically preserves speech characteristics better than naive global resampling.

In the implemented system, duration scaling begins by detecting approximately pitch-synchronous marks within the selected speech segment. This is done using normalized autocorrelation over short analysis frames. For each frame, the algorithm searches for a plausible pitch period within a voiced-speech range and then refines the corresponding mark to a nearby strong waveform extremum. This refinement step helps align synthesis to the local waveform structure more accurately.

After pitch marks are detected, local pitch periods are estimated from the spacing between neighboring marks. These periods are then used to construct synthesis locations for the output segment. For duration expansion, output pitch-mark positions are placed farther apart, causing waveform grains to be overlap-added over a longer total interval. For duration compression, output positions are placed closer together while source pitch marks are traversed more aggressively, effectively shortening the segment.

The waveform grains used for synthesis are extracted around each detected pitch mark using neighboring pitch marks to define the grain boundaries. Each grain is windowed with a Hann window before synthesis. The modified output is then generated by overlap-adding these grains at the new synthesis positions. A corresponding synthesis-weight array is accumulated and used for normalization so that overlapping regions are averaged appropriately rather than summed without correction.

The implementation also includes several practical safeguards. If too few pitch marks are detected, which may occur for very short or poorly voiced segments, the system falls back to simple linear interpolation resampling rather than forcing PSOLA on an unreliable input. After synthesis, the output is normalized if necessary to prevent excessive peak amplitude, and a small DC offset is removed.

Within the broader framework described earlier, this TD-PSOLA module is called through the duration-scaling interface. The extracted waveform segment and requested scale factor are passed into the algorithm, and the resulting modified segment is then returned for stitching back into the original waveform. For target-duration edits, the desired duration is converted externally into an equivalent scale factor so that both user-facing edit modes share the same underlying processing method.

Although this method successfully performs duration modification on voiced speech segments, its quality depends strongly on pitch-mark accuracy and on the characteristics of the selected segment. Imperfect pitch-mark placement, mixed voiced/unvoiced content, or difficult segment boundaries can lead to audible artifacts such as roughness, discontinuities, or a warbled quality in the modified output.

6 Evaluation Methodology

Evaluation of the project focused on both software correctness and output quality. Because the project is both a signal-processing system and a software framework, it was important to assess not only whether the edited speech exhibited the intended duration change, but also whether the implementation behaved correctly under different edit specifications and processing scenarios.

From a software-validation perspective, the framework was tested using automated tests covering key pipeline behavior, including waveform loading, segment extraction, stitching, CSV-based batch edit parsing, target-duration handling, remapping of later batch edits relative to the original input timeline, and rejection of overlapping edit requests. These tests were used to verify the correctness and robustness of the framework independently of the final speech-modification quality.

From a signal-analysis perspective, a separate evaluation script was developed to compare original and modified audio files. At the full-signal level, this script generates an overlaid waveform comparison, separate spectrograms for the original and modified signals, and a text-based metrics summary. The full-signal metrics include sample rate, number of samples, total duration, duration ratio between modified and original audio, RMS amplitude, and peak absolute amplitude.

In addition to full-signal analysis, the script supports region-focused evaluation of a selected edited interval. When the edited segment boundaries are provided, the script extracts the specified time region and generates a localized waveform comparison around that interval, including an optional time margin to provide context before and after the edit. It also generates region-specific spectrograms for both the original and modified signals and writes a separate region metrics file containing values such as requested interval duration, original and modified region sample counts, actual region durations, RMS amplitude, and peak absolute amplitude.

These outputs were used to evaluate whether the selected segment had undergone the intended temporal modification and to inspect how the spectral and waveform structure changed after processing. In particular, the region-focused plots were useful for examining the edited portion directly rather than relying only on whole-signal visualizations.

Finally, qualitative listening evaluation was used alongside the generated plots and metrics. Because the project goal is perceptual duration modification of speech rather than exact waveform reconstruction, listening-based assessment was used to judge intelligibility, preservation of speaker character, and the presence of audible artifacts such as roughness, discontinuities, or warbling in the modified region. Together, these software tests, visual analyses, summary metrics, and listening observations formed the basis of the final project evaluation.

7 Results

The final system successfully implemented an end-to-end workflow for user-directed duration modification of selected speech segments. The framework was able to load an input mono WAV file, accept an edit request,

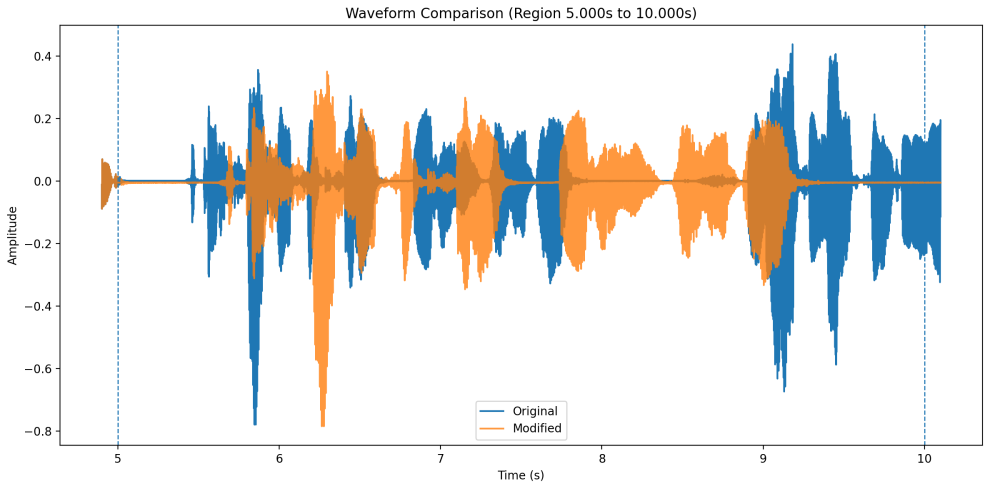

Figure 1: Region-focused waveform comparison for the 5 s to 10 s edited interval. The modified waveform diverges from the original after the onset of the duration modification, consistent with temporal expansion of the selected speech segment.

extract the specified segment, apply TD-PSOLA-based duration modification, reconstruct the waveform, and write the edited result to a new output file.

The primary demonstration case used throughout development involved a speech recording of a woman reading a narrative passage. In the relevant portion of the input, the utterance included the sentence: "He had little enough to break or bite, and once, when there was a great famine in the land, he could hardly procure even his daily bread. And as he lay thinking in his bed one night, he sighed and said to his wife, 'what will become of us'." The demonstration command used for this case was:

```
python main.py examples/test.wav --output examples/output_test.wav --t1 5 --t2 10 --scale 1.5
```

This command applies duration expansion to the segment from 5 s to 10 s in the original waveform using a scale factor of 1.5. In perceptual terms, the modification begins around the portion of the utterance starting with "he could hardly procure...", and the selected region is stretched relative to the original signal.

Evaluation of this demonstration case showed that the system performed the requested duration modification correctly at the file level. The original recording had a total duration of 15.22 s, while the modified output had a duration of 17.72 s, corresponding to an increase of 2.50 s. This matches the requested edit: the selected region was 5 s long, and scaling that region by a factor of 1.5 increases its duration to 7.5 s, producing a net increase of 2.5 s in the full output waveform.

Signal-level summary statistics also remained stable after modification. The RMS amplitude changed only slightly, from 0.0734 in the original signal to 0.0714 in the modified signal, and the peak absolute amplitude remained nearly unchanged, increasing only from 0.7798 to 0.7852. These values indicate that the duration-modification process changed the temporal structure of the selected region without causing large overall amplitude distortion.

Region-focused evaluation around the 5 s to 10 s edit interval also showed clear local divergence between the original and modified waveforms, as shown in Figure 1. The corresponding modified spectrogram, shown in Figure 2, indicates that the broad time-frequency structure of the speech was preserved despite the temporal expansion. In particular, the waveform comparison around the edited region made it visually apparent that the modified signal no longer aligns with the original after the onset of the edit, which is consistent with the inserted temporal expansion in that segment. Region-focused spectrograms likewise showed that the modified output retained the broad time-frequency structure of speech while reflecting the temporal changes introduced by the duration-modification process.

Perceptually, the modified result remained intelligible, and the speech was still recognizable as the same speaker with much of the original tone and inflection preserved. At the same time, the output was not

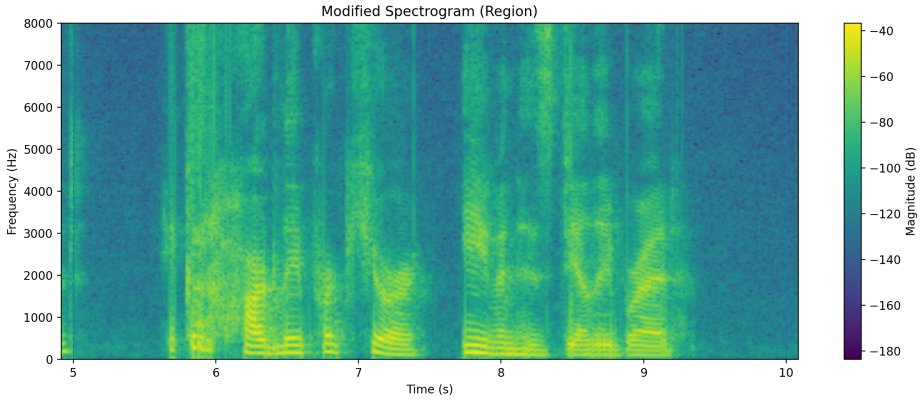

Figure 2: Region-focused spectrogram of the modified signal around the edited interval. The modified output preserves the broad time-frequency structure of the speech while reflecting the temporal expansion introduced by the edit.

artifact-free. Listening evaluation indicated audible degradation in the modified segment, particularly a degree of roughness or warbling introduced by the PSOLA-based duration-modification process. These artifacts were noticeable enough that the output clearly sounded modified, but they did not prevent the speech from remaining understandable.

Beyond the main perceptual demonstration, the software framework also successfully supported additional functionality such as target-duration editing, CSV-based batch edit input, timeline remapping for multiple edits relative to the original waveform, and rejection of overlapping edit regions. These features were important results from the software-engineering perspective, even though the primary qualitative evaluation focused on the single-edit speech demonstration described above.

Overall, the strongest result of the project is that it produced a complete and functional duration-editing framework rather than only an isolated signal-processing routine. While the perceptual quality of the modified speech still leaves room for improvement, the final system demonstrates that arbitrary user-specified segment duration changes can be represented, validated, processed, and evaluated within a coherent and extensible architecture.

8 Discussion and Limitations

A major strength of this project is that it produced more than an isolated signal-processing routine. In addition to the core duration-modification algorithm, the project resulted in a complete software framework for speech-segment editing, including WAV input/output, edit validation, single-edit and batch-edit workflows, reconstruction logic, automated testing, and evaluation tooling. From a software-engineering perspective, this broader system design is one of the strongest outcomes of the project.

At the same time, the perceptual quality of the final edited speech reveals important limitations of the current implementation. Although the system successfully performed the requested duration modification and produced intelligible output, the edited speech was not artifact-free. In the primary demonstration case, the modified segment remained understandable and preserved much of the speaker's tone and identity, but it also exhibited audible roughness and warbling. This indicates that while the TD-PSOLA-based approach was effective as a proof of concept, its current implementation does not yet achieve consistently natural-sounding speech modification.

One likely source of these artifacts is the difficulty of accurate pitch-mark estimation. The implemented method detects approximate pitch-synchronous marks using short-time autocorrelation and then refines them to nearby waveform extrema. While this approach is reasonable and lightweight, it is still sensitive to pitch-estimation errors, local waveform irregularities, and transitions between voiced and unvoiced speech. If pitch marks are not aligned well, overlap-add synthesis can introduce discontinuities or unnatural modulation in the edited output.

A second limitation is that the evaluation methodology, while useful, was not exhaustive. The project included waveform comparisons, spectrogram visualizations, and basic signal-level metrics such as duration, RMS amplitude, and peak amplitude, together with qualitative listening-based assessment. These methods were sufficient to verify that the requested duration change occurred and that the broad structure of the speech signal was preserved, but they do not constitute a full perceptual evaluation. More advanced speech-quality metrics or broader listening studies would be needed to assess naturalness more rigorously.

Another limitation is that the primary qualitative evaluation focused on one main single-edit demonstration case. This case was useful because it clearly illustrated the intended behavior of the framework, but a broader set of examples would provide stronger evidence of robustness across different speakers, phonetic content, and edit magnitudes. Similarly, while batch editing and target-duration functionality were implemented and validated within the software framework, the most detailed perceptual analysis in this report centered on a single stretched segment rather than on a broader collection of edit scenarios.

Finally, the project necessarily balanced algorithmic refinement against software completeness. Significant effort was devoted to building a modular architecture, validating user inputs, supporting batch workflows, and constructing testing and evaluation tools. This improved the overall completeness and usability of the project, but it also limited the time available for more extensive tuning of the TD-PSOLA stage itself. As a result, the software framework is more mature than the perceptual polish of the final speech output.

Future work could improve the system in several ways. More robust pitch-mark detection, smoother boundary handling during reinsertion, and improved treatment of voiced/unvoiced transitions would likely reduce audible artifacts. An additional direction for future work would be support for contour-based duration control, in which the time-scaling applied within a selected segment varies according to a user-specified contour rather than remaining uniform across the entire interval. This would make the system more flexible and could enable more natural or expressive modifications than a single global scaling factor alone. Additional evaluation across multiple speech examples and edit configurations would also strengthen confidence in the method. More broadly, the modular architecture developed in this project provides a good foundation for experimenting with refined PSOLA variants or alternative duration-modification techniques in future work.

9 Conclusion

This project developed a modular software framework for arbitrary duration modification of selected segments within a speech waveform. The system supports user-specified edits through either scale-factor-based or target-duration-based input, and provides an end-to-end pipeline including WAV input/output, edit validation, segment extraction, duration modification, waveform reconstruction, batch processing, and evaluation support.

The core duration-modification method selected for the project was a TD-PSOLA-based approach. Within the completed framework, this method serves as the algorithmic mechanism used to lengthen or shorten selected speech regions while attempting to preserve perceptually important characteristics of the original signal.

The final results show that the system successfully performs targeted duration edits and produces intelligible modified speech, while also revealing the practical challenges of artifact reduction in real speech processing. In the primary demonstration case, the requested temporal expansion was achieved correctly at the file level, and the output remained recognizable as the original speaker, although audible roughness and warbling were still present in the edited region.

Overall, the project achieved its primary goal of building a functional and extensible duration-modification framework for speech. In addition to satisfying the immediate project objectives, the resulting architecture provides a strong foundation for future refinements to pitch-mark detection, overlap-add synthesis quality, contour-based duration control, and broader perceptual evaluation.